题目相关

第一章

1.操作系统是对计算机资源进行管理的软件

2.批处理作业必须具有作业控制信息

3.所有分时系统都具有人机交互功能，这也是分数系统的优点

4.分时系统无法优先处理紧急事务，试试系统可以做到立即响应

5.分时是通过多程序或者多任务将计算机资源分配给许多用户，用户无法独占

6.最基本的特征：并发、共享（两者互为存在条件）

7.并发是指若干事件在同一时间间隔内发生

8.分时系统没有及时响应的功能

9.实时系统的交互性是人与系统的交互，分时系统的交互性是人与人之间共享资源

10.原语：操作系统的最底层，最接近硬件的部分，这些程序的运行具有原子性，操作只能一起呵成，运行时间短且调用频繁

11.CPU处于核心态时候可以执行除"访管"指令的全部指令

访管指令是在用户态使用的，该指令的含义是用户资源进入核心态

核心态下可以执行特权和非特权指令

12.用户态到核心态的转换由硬件完成

第二章

1.程序和进程的区别：进程是动态的，程序是静态的。程序是有序代码的集合，进程是程序的执行。进程是暂时的，程序是永久的。通过多次执行，一个程序可以对应多个进程。通过调用关系，一个进程可包含多个程序。

操作系统

一、操作系统概述

1.1操作系统的概念

- **操作系统（OS）：**
  - 控制和管理整个计算机系统的**硬件和软件**资源，并合理地组织调度计算机的工作和资源的分配。
  - 提供给用户和其他软件方便的接口和环境。
- 是计算机系统中**最基本的系统软件。**
- **作为系统资源的管理者:**
  - 提供的功能：

    管理与配置内存、决定系统资源供需的优先次序、控制输入与输出设备、操作网络与管理文件系统等，也提供一个让用户与系统交互的操作页面。

        - 处理机管理

          进程控制，进程同步，进程通信，死锁处理处理机调度等
        - 存储器管理

          内存分配，地址映射，内存保护与共享和内存扩充等
        - 文件管理

          文件存储空间的管理，目录管理及文件读写管理和保护
        - 用户接口
        - 设备管理

          缓冲管理，设备分配，设备处理和虚拟设备
    - 目标：
        - 高效，安全

> 执行一个程序前需要将该程序放到内存中，才能够被CPU处理

GUI：图形化用户接口

联机命令接口=交互式命令接口：用户说一句，系统执行一句

脱机命令接口=批处理命令接口：用户说一堆，系统执行一堆

程序接口=广义指令：在程序中通过**系统调用**来使用（普通用户无法直接使用，只能通过代码间接使用）

操作系统的目标：对硬件机器的拓展

裸机：没有任何软件支持的计算机

虚拟机=扩充机器：覆盖了软件的机器

- **操作系统的特征**：

  最基本的特征：并发、共享（两者互为存在条件）

    - **并发**：两个或多个事件在同一时间**间隔**内发生。这些事件在**宏观上是同时发生**的，但在**微观上是交替发生**的。

      操作系统的并发性指在计算机系统中“同时”运行着多个程序，宏观上看同时运行，微观上看交替运行。

      操作系统和程序并发是一起诞生的。

      单核CPU：同一时间只能执行一个程序，各个程序只能并发执行

      多核CPU：同一时刻可以同时执行多个程序，多个程序可以并行执行

      > 4核CPU可以并行地执行4个程序

      并发性是操作系统一个最基本的特性
    - **并行**：两个或多个事件在同一时刻内同时发生
    - **共享**：资源共享，系统中的资源可供内存中多个并发执行的进程共同使用
    - 互斥共享方式：一个时间段内只允许一个进程访问该资源
        - 同时共享方式：允许某一个时间段内有多个进程“同时”对它们进行访问

      > 同时是宏观上看，微观上可能在交替进行

    - **虚拟**：把一个物理的实体变为若干个逻辑上的对应物，物理实体是实际存在的，对应物是用户感受到的
        - 空分复用技术（虚拟存储技术）

- 时分复用技术（虚拟处理器）

> 一个程序需要放入内存并且给它分配CPU才能执行

没有并发性就没有虚拟性

- **异步：**在多道程序环境下，允许多个程序并发执行，但由于资源有限，进程的执行不是一贯到底的，而是走走停停，已不可预知的速度向前推进

  只有系统拥有并发性才可能拥有异步性

1.2操作系统的发展与分类

- **手工操作阶段：**人机速度矛盾批处理阶段（操作系统开始出现，此阶段无操作系统缺点）
  - 欠点：用户独占全机，人机速度矛盾导致资源利用率极低
- **批处理阶段：**
  - **单道批处理系统：**引用**脱机输入/输出技术**（外围机+磁带），并且由**监督程序**负责控制作业的输入输出
    - 利点：缓解了人机速度矛盾
    - 欠点：内存中仅能有一道程序运行，CPU有大量时间是在空闲等待I/O完成，**资源利用率低**
  - **多道批处理系统：**操作系统的正式诞生
    - 利点：多道程序**操作系统的正式诞生**，共享计算机资源，资源利用率大幅提升
    - 欠点：用户响应时间长，**没有人机交互功能**
- **分时操作系统：**计算机用**时间片**为单位**轮流为各个用户/作业服务**，多个用户可以通过终端与计算机进行交互（出现人机交互）
  - 利点：用户请求可以被即时响应，解决了人机交互问题
  - 欠点：不能优先处理一些紧急任务（不能插队）
- **实时操作系统：**接受到外部信号后及时处理，并且需要在严格的时限内处理完事件
  - 利点：能够优先响应一些紧急任务（可以插队），某些紧急任务不需时间片排队
  - 主要特点：及时性和可靠性
  - 分类：
    - 硬实时系统：必须在严格时间内处理完成

      > 例如：导弹控制系统，自动驾驶系统

    - 软实时系统：可以偶尔违反规定

      > 例如12306火车订票

- 网络操作系统：实现网络中各种资源的共享（如文件共享）和各台计算机之前的通信
- 分布式操作系统：分布性和并行性，任何任务都可以分布在这些计算机上，由它们并行、协同完成这个任务
- 个人计算机操作系统：如WindowsXP，MacOS，方便个人使用

1.3操作系统的运行机制

- **应用程序和内核程序**

程序运行的过程其实就是CPU执行一条一条机器指令的过程

应用程序：普通程序员写的

内核程序：负责实现操作系统的人所实现

操作系统内核（内核）：由很多内核程序组成

内核是操作系统最重要最核心的部分，也是**最接近硬件的部分**，一个操作系统只需要有内核就可以

- **特权指令和非特权指令**

在CPU设计和生产的时候就划分了特权指令和非特权指令

应用程序只能使用“非特权指令”

操作系统内核可以让CPU指向一些“特权指令”

- **CPU的两种状态：内核态和用户态**

处于内核态，说明正在运行内核程序，可以执行特权指令

处于用户态，说明正在运行应用程序，只能执行非特权指令

> CPU中有一个寄存器叫做**程序状态字寄存器（PSW），其中有二进制位，1：内核态，2：用户态

内核态=核心态=管态

用户态=目态

内核态->用户态：执行一条特权指令，修改PSW的标志位为“用户态”

用户态->内核态：由中断引法，硬件自动完成变态过程，操作系统强行夺回CPU的使用权

> 只要是需要操作系统介入的地方，都需要触发中断信号

- **中断和异常：**

  中断的作用：让操作系统内核夺回CPU使用权的唯一途径

  中断的类型：

  - 内中断（异常、例外）：与当前执行的指令有关，中断信号来源于CPU内部（自愿中断，指令中断）
    - 陷入、陷阱：由陷入指令引发，是应用系统故意引发
    - 故障：由错误条件引发，可能被内核程序修复，修复后会把CPU使用权还给应用程序
    - 终止：由致命错误引发，不能被内核程序修复，不会把CPU使用权还给应用程序

    > 例：1.在用户态执行特权指令
    >
    > 2.执行的指令是非法的
    >
    > 3.执行陷入指令（不是特权指令）：应用程序主动将CPU控制权还给操作系统内核，系统调用是通过陷入指令（trap指令、访管指令）完成的

  - 外中断（中断）：与当前执行的指令无关，中断信号来源于CPU外部（外设请求，人工干预）
    - 时钟中断
    - I/O中断请求

    > 例：I/O中断——由输入/输出设备发来的中断信号

  中断机制的基本实现原理：不同的中断信号，需要用不同的中断处理程序来进行处理，检测到中断信号后会根据信号类型查询**“中断向量表”**用来查找相应中断处理程序在内存中的存放位置

- **系统调用**

  系统调用与库函数区别:

  - 有的库函数是对系统调用的进一步封装
  - 有的库函数没有使用系统调用

程序接口由一组系统调用组成

系统调用是操作系统提供给应用程序（程序员）使用的接口，应用程序可以听过系统调用来请求获得操作系统内核的服务

发出系统调用是在用户态，而对系统调用的相应处理在核心态进行



1.4操作系统的体系结构

- **大内核和微内核**
  - 大内核=宏内核=单内核：将操作系统的主要功能模块都作为系统内核，运行在核心态
    - 利点：高性能
    - 欠点：内核代码庞大，结构混乱，难以维护
  - 微内核：只把最基本的功能保留在内核
    - 利点：内核功能少，结构清晰，方便维护
    - 欠点：需要频繁地在核心态和用户态之间切换，性能低

> 典型大内核：Linux、UNIX
>
> 微内核：Windows NT

	特性、思想	优点	缺点
分层结构	内核分多层，每层可单向调用更低一层提供的接口	1. 便于调试和验证，自底向上逐层调试验证	1. 仅可调用相邻低层，难以合理定义各层的边界
		2. 易扩充和易维护，各层之间调用接口清晰固定	2. 效率低，不可跨层调用，系统调用执行时间长
模块化	将内核划分为多个模块，各模块之间相互协作。 内核 = 主模块+可加载内核模块 主模块：只负责核心功能，如进程调度、内存管理 可加载内核模块：可以动态加载新模块到内核，而无需重新编译整个内核	1. 模块间逻辑清晰易于维护，确定模块间接口后即可多模块同时开发	1. 模块间的接口定义未必合理、实用
		2. 支持动态加载新的内核模块（如：安装设备驱动程序、安装新的文件系统模块到内核），增强OS适应性	
		3. 任何模块都可以直接调用其他模块，无需采用消息传递进行通信，效率高	2. 模块间相互依赖，更难调试和验证
宏内核（大内核）	所有的系统功能都放在内核里（大内核结构的OS通常也采用了"模块化"的设计思想）	1. 性能高，内核内部各种功能都可以直接相互调用	1. 内核庞大功能复杂，难以维护
			2. 大内核中某个功能模块出错，就可能导致整个系统崩溃
微内核	只把中断、原语、进程通信等最核心的功能放入内核。进程管理、文件管理、设备管理等功能以用户进程的形式运行在用户态	1. 内核小功能少、易于维护，内核可靠性高	1. 性能低，需要频繁的切换 用户态/核心态。 用户态下的各功能模块不可以直接相互调用，只能通过内核的"消息传递"来间接通信
		2. 内核外的某个功能模块出错不会导致整个系统崩溃	2. 用户态下的各功能模块不可以直接相互调用，只能通过内核的"消息传递"来间接通信
外核（exokernel）	内核负责进程调度、进程通信等功能，外核负责为用户进程分配未经抽象的硬件资源，且由外核负责保证资源使用安全	1. 外核可直接给用户进程分配"不虚拟、不抽象"的硬件资源，使用户进程可以更灵活的使用硬件资源	1. 降低了系统的一致性
		2. 减少了虚拟硬件资源的"映射层"，提升效率	2. 使系统变得更复杂

1.5虚拟机

使用虚拟化技术，将一台物理机器虚化为多台虚拟机器，每个虚拟机都可以独立运行一个操作系统

虚拟机=虚拟机管理程序=虚拟机监控程序

两类虚拟机管理程序（VMM）的对比

	第一类VMM	第二类VMM
对物理资源的控制权	直接运行在硬件之上，能直接控制和分配物理资源	运行在Host OS之上，依赖于Host OS为其分配物理资源
资源分配方式	在安装Guest OS时，VMM要在原本的硬盘上自行分配存储空间，类似于"外核"的分配方式，分配未经抽象的物理硬件	GuestOS 拥有自己的虚拟磁盘，该盘实际上是 Host OS 文件系统中的一个大文件。GuestOS分配到的内存是虚拟内存
性能	性能更好	性能更差，需要HostOS作为"中介"
可支持的虚拟机数量	更多，不需要和 Host OS 竞争资源，相同的硬件资源可以支持更多的虚拟机	更少，Host OS 本身需要使用物理资源，Host OS 上运行的其他进程也需要物理资源
虚拟机的可迁移性	更差	更好，只需导出虚拟机镜像文件即可迁移到另一台 HostOS 上，商业化应用更广泛
运行模式	第一类VMM运行在最高特权级（Ring 0），可以执行最高特权的指令。	第二类VMM部分运行在用户态、部分运行在内核态。GuestOS 发出的系统调用会被 VMM 截获，并转化为 VMM 对 HostOS 的系统调用

二、进程管理

2.1进程概念、组成、特征

程序：静态的，存放在磁盘内地可执行文件，是一系列的指令集合。

进程：动态的，是程序的一次执行过程

同一个程序多次执行回对应多个进程

PID：每个进程会有一个**不重复的，唯一的**进程ID

操作系统需要对各个并发运行的进程进行管理，但凡管理时所需要的信息，都会被放在**PCB（进程控制块）**中



操作系统对进程进行管理工作所需的信息都存在PCB中

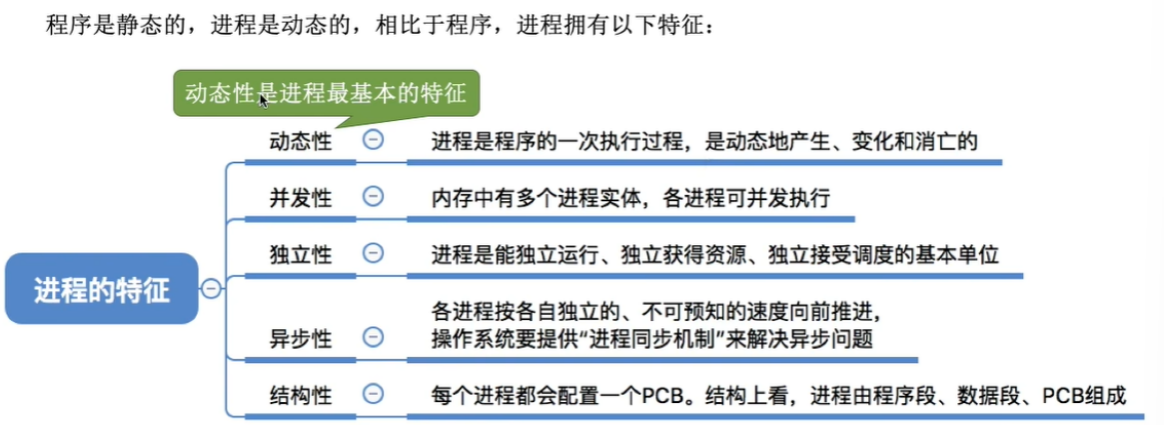
- **进程实体（进程映像）的组成：**
  - PCB
  - 程序段：程序的代码
  - 数据段：运行中产生的各种数据

    PCB是给操作系统用的

    程序段和数据段是给进程自己用的

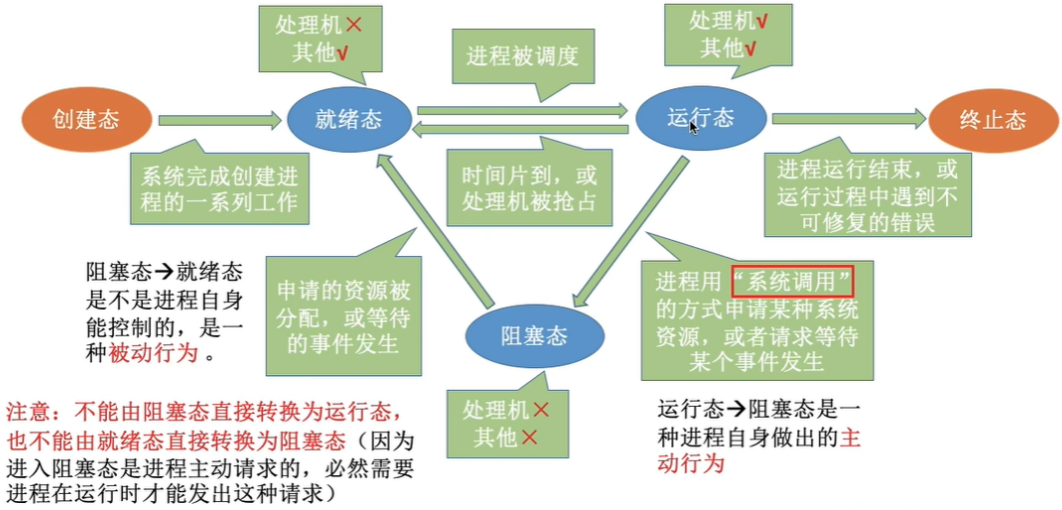
进程（动态）是进程实体（静态）的运行过程，是系统进行**资源分配和调度的**一个基本单位，是操作系统结构的基础

PCB是进程存在的唯一标志



2.2进程的状态和转换

- 进程的状态



- **三种基本状态**：
  - 运行态：占有CPU，并在CPU上运行
  - 就绪态：具备运行条件，但是没有空闲CPU，暂时无法运行
  - 阻塞态=等待态：因等待某一件事情而暂时不能运行

进程PCB中，会有一个变量state来表示进程的当前状态

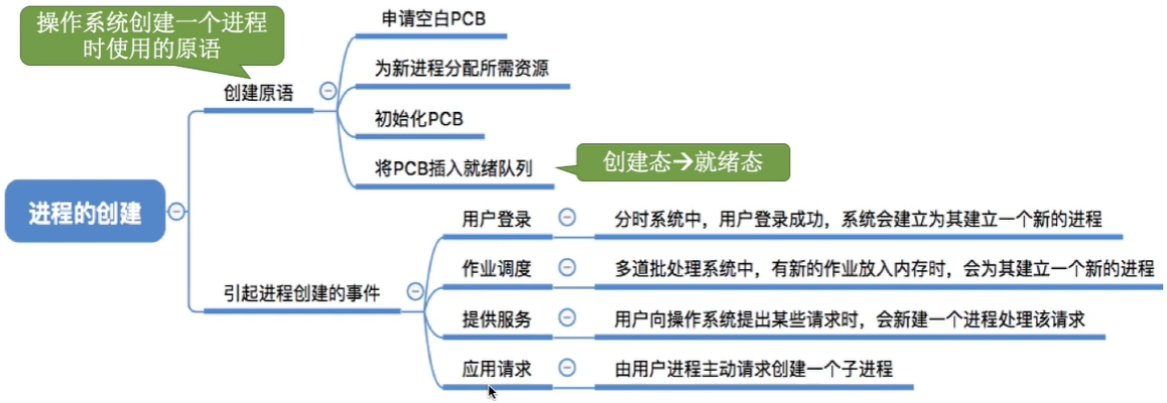
- **进程的组织**
  - 链接方式
  - 索引方式

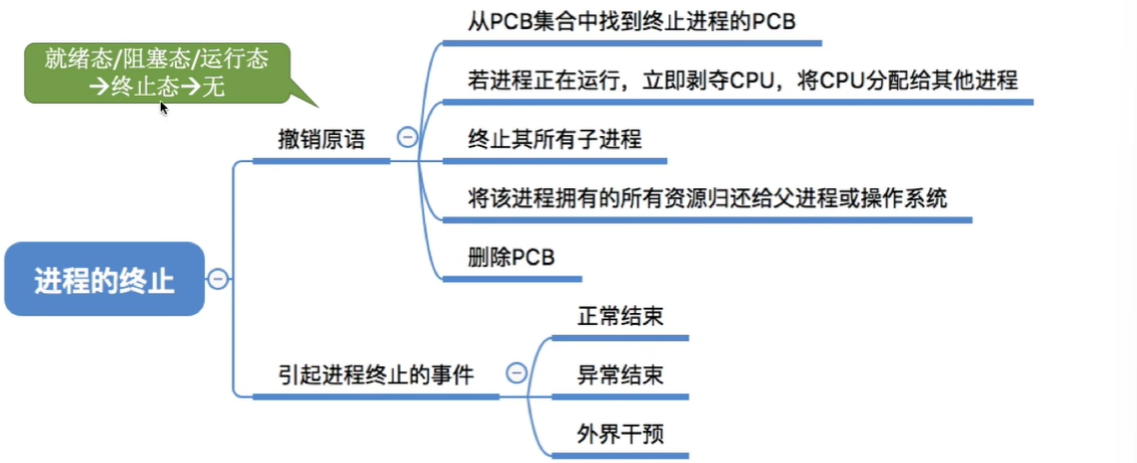
2.3进程控制

进程控制：进行状态转换

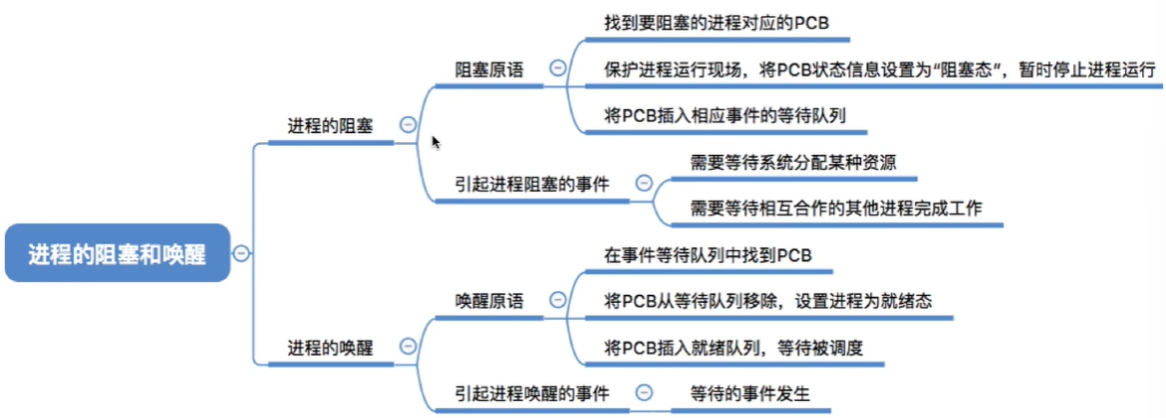
原语：执行具有原子性，运行必须一气呵成不可以中断

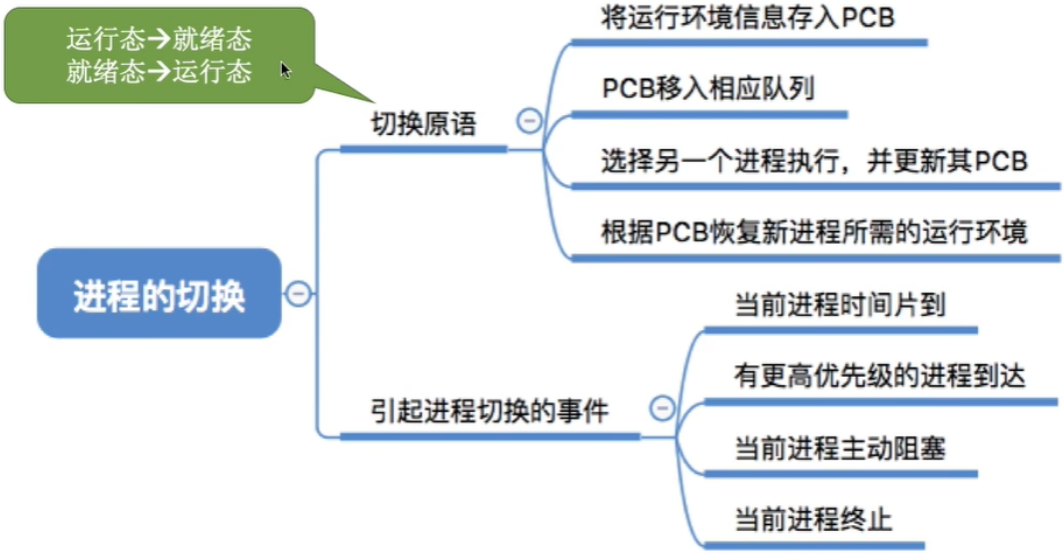
可以用"关中断指令"和"开中断指令"这两个特权指令实现原子性





进程之间的关系是树形结构





阻塞和唤醒要成对出现

2.4进程间通信（IPC）

各个进程的内存地址空间相互独立

- **共享存储**：（互斥访问共享空间）
  - 基于数据结构的共享，速度慢限制多，低级通信方式
  - 基于存储区共享，操作系统在内存中划出一块共享存储区，速度快，高级通信方式

进程间的数据交换以格式化的消息为单位

- **消息传递：**
  - 直接通信
- 间接通信=信箱通信方式
- **管道通信**：只能单向传递（半双工通信）

  各进程需要互斥访问管道

要实现双向通信则需要设置两个管道

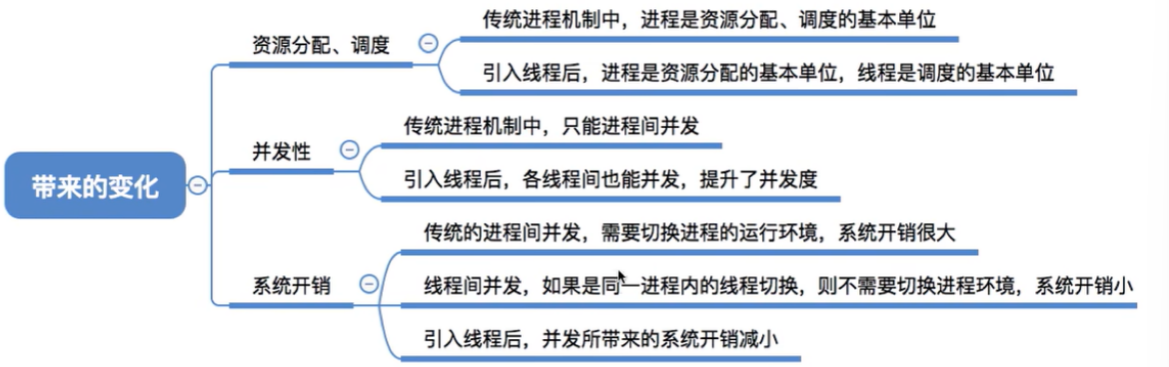
一个管道运行多个写进程，一个读进程

2.5线程

线程是一个**基本的CPU执行单元**，也是**程序执行流的最小单位**

进程内的各个**线程也可以并发**，提高了系统的并发度

引入线程后，进程只作出CPU外的系统资源的分配单位



- **线程的属性**
  - **线程是操作系统进行资源分配的最小单位**
  - 被包含在进程之中，是进程中的实际运作单位
  - 多CPU计算机中，各个线程可占用不同的CPU
  - 每个线程都有一个线程ID、线程控制块(TCB)
  - 线程也有就绪、阻塞、运行三种基本状态
  - 线程几乎不拥有系统资源
  - 同一进程的不同线程间共享进程的资源
  - 由于共享内存地址空间，同一进程中的线程间通信甚至无需系统干预
  - 同一进程中的线程切换，不会引起进程切换
  - 不同进程中的线程切换，会引起进程切换
  - 切换同进程内的线程，系统开销很小
- 切换进程，系统开销较大

- **线程的实现方式**
  - 用户级线程：线程管理工作由应用程序负责
    - 利点：系统开销低，效率高
    - 欠点：一个线程被阻塞，其他的都会被阻塞，不可能在多核处理机上并行运行
  - 内核级线程=内核支持的线程：由操作系统内核完成
    - 利点：并发性强
    - 欠点：系统开销大
  - 多线程模型
    - 一对一：一个用户级线程映射到一个内核级线程
      - 优:各个线程可分配到多核处理机并行执行，并发度高
      - 缺:线程管理都需要操作系统支持，开销大
    - 多对一：多个用户级线程映射到一个内核级线程
      - 优:线程管理开销小效率高
      - 缺:一个线程阻塞会导致整个进程都被阻塞(并发度低)
    - 多对多：n个用户级线程映射到m个内核级线程(n>=m)
      - 集二者之所长

内核级线程才是处理机分配的单位

- 线程的状态和转换和进程一样

三、处理机调度与死锁

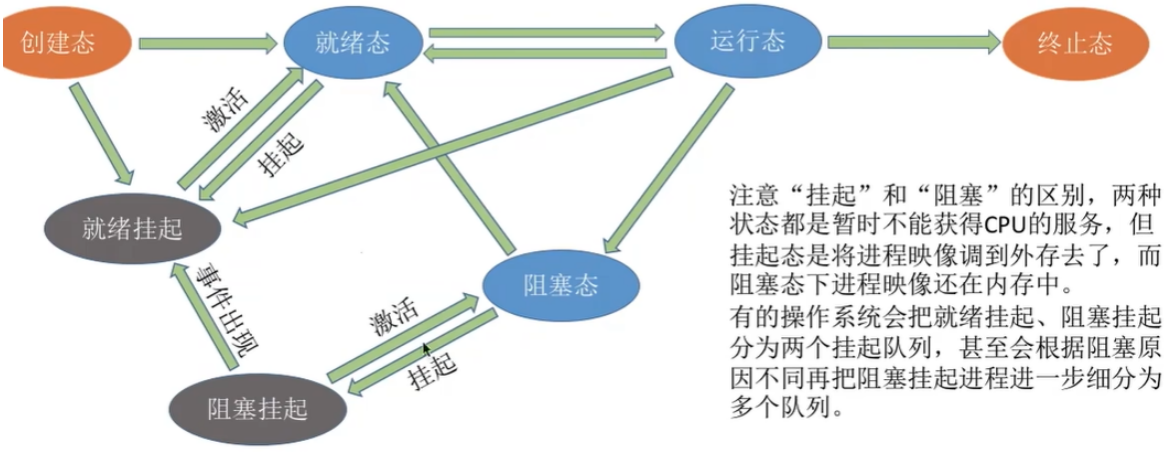
3.1处理机调度

在多道程序系统中，调度的实质是一种资源分配，处理机调度是对处理机资源进行分配。

处理机调度算法是指根据处理机分配策略所规定的处理机分配算法。

- 高级调度=作业调度：每个作业只调入一次，调出一次
- 低级调度=进程调度=处理机调度：最基本的调度，频率很高
- 中级调度=内存调度：决定调入内存的处于挂起状态的进程，频率高于高级调度

暂时调到外存等待的进程状态为挂起状态（挂起态，suspend）
挂起态又可以进一步细分为就绪挂起、阻塞挂起两种状态
五状态模型 → 七状态模型



	要做什么	调度发生在..	发生频率	对进程状态的影响
高级调度（作业调度）	按照某种规则，从后备队列中选择合适的作业将其调入内存，并为其创建进程	外存→内存（面向作业）	最低	无→创建态→就绪态
中级调度（内存调度）	按照某种规则，从挂起队列中选择合适的进程将其数据调回内存	外存→内存（面向进程）	中等	挂起态→就绪态（阻塞挂起→阻塞态）
低级调度（进程调度）	按照某种规则，从就绪队列中选择一个进程为其分配处理机	内存→CPU	最高	就绪态→运行态

进程在内核临界区中不能进行调度和切换

内核临界区：访问某种内核数据结构

- **进程调度的任务：**
  - 保存处理机的现场信息
  - 按某种算法选取进程
  - 把处理器分配给进程
- **进程调度机制**：为了实现进程调度，在进程调度机制中，应具有如下三个基本部分
  - 排队器
  - 分派器
  - 上下文切换器
- **调度的方式：**
  - 非剥夺调度方式=非抢占方式：直至该进程完成，或发生某事件而被阻塞时，才把处理机分配给其它进程
  - 剥夺调度方式=抢占方式 ：允许调度程序根据某种原则，去暂停某个正在执行的进程，将已分配给该进程的处理机重新分配给另一进程

在现代OS中广泛采用抢占方式

- **处理机调度算法**
  - 提高资源利用率：

$$\text{CPU 的利用率} = \frac{\text{CPU 有效工作时间}}{\text{CPU 有效工作时间} + \text{CPU 空闲等待时间}}$$

  - 公平性：

    公平性是指应使诸进程都获得合理的CPU 时间，不会发生进程饥饿现象
  - 平衡性：

    为使系统中的CPU和各种外部设备都能经常处于忙碌状态，调度算法应尽可能保持系统资源使用的平衡性。
  - 策略强制执行
- **批处理系统的目标**
  - 平均周转时间短
  - 系统吞吐量高

    由于吞吐量是指在单位时间内系统所完成的作业数，因而它与批处理作业的平均长度有关
  - 处理机利用率高

- **分时系统的目标**
  - 响应时间快
  - 均衡性
- **实时系统的目标**
  - 截止时间的保证
  - 可预测性
- **作业与作业调度：** 在多道批处理系统中，作业是用户提交给系统的一项相对独立的工作
- **批处理系统中的作业**
  - 作业和作业步
- **作业控制块：**为了管理和调度作业，在多道批处理系统中，为每个作业设置了一个作业控制块JCB，它是作业在系统中存在的标志，其中保存了系统对作业进行管理和调度所需的全部信息
- **作业运行的三个阶段和三种状态**

  作业从进入系统到运行结束，通常需要经历收容、运行和完成三个阶段。相应的作业也就有“后备状态”、“运行状态”和“完成状态”

3.1.1调度算法

- **先来先服务(FCFS)调度算法**

  FCFS是最简单的调度算法，该算法既可用于作业调度，也可用于进程调度

  当在作业调度中采用该算法时，系统将按照作业到达的先后次序来进行调度

- **短作业优先(SJF)的调度算法**
  - 短作业优先算法：SJF算法是以作业的长短来计算优先级，作业越短，其优先级越高
  - SJF算法可以分别用于作业调度和进程调度
  - 缺点：
    - 必须预知作业的运行时间
    - 对长作业非常不利，长作业的周转时间会明显地增长
    - 在采用SJF算法时，人—机无法实现交互
    - 不能保证紧迫性作业能得到及时处理
- **优先级调度算法（PSA）**：把处理机分配给就绪队列中优先级最高的进程
  - 非抢占式优先级调度算法
  - 抢占式优先级调度算法
- **优先级的类型**
  - 静态优先级：静态优先级是在创建进程时确定的，在进程的整个运行期间保持不变
    - 进程类型
    - 进程对资源的需求
    - 用户要求
  - 动态优先级：动态优先级是指在创建进程之初，先赋予其一个优先级，然后其值随进程的推进或等待时间的增加而改变，以便获得更好的调度性能
- **高响应比优先调度算法**

  既考虑了作业的等待时间，又考虑作业运行时间的调度算法

$$优先权 = \frac{等待时间 + 要求服务时间}{要求服务时间}$$

$$R_P = \frac{\text{等待时间} + \text{要求服务时间}}{\text{要求服务时间}} = \frac{\text{响应时间}}{\text{要求服务时间}}$$

- **轮转调度算法**
  - 系统将所有的就绪进程按FCFS策略排成一个就绪队列
  - 进程切换时机：
    - 若一个时间片尚未用完，正在运行的进程便已经完成，就立即激活调度程序，将它从就绪队列中删除，再调度就绪队列中队首的进程运行，并启动一个新的时间片
    - 在一个时间片用完时，计时器中断处理程序被激活。如果进程尚未运行完毕，调度程序将把它送往就绪队列的末尾

在轮转算法中，时间片的大小对系统性能有很大的影响

- **多队列调度算法**

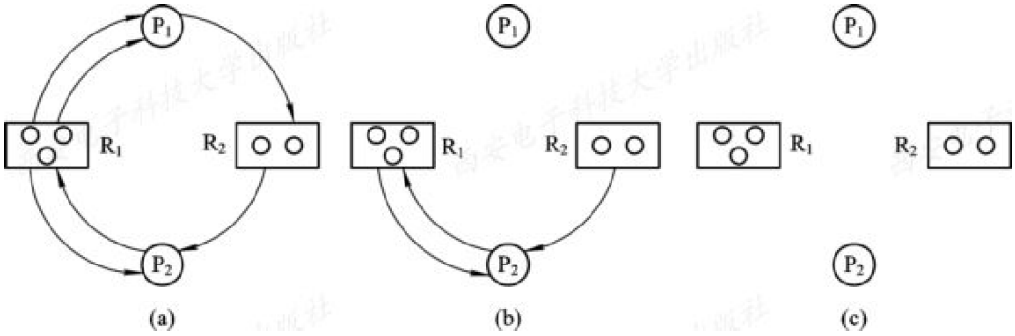
如前所述的各种调度算法，尤其在应用于进程调度时，由于系统中仅设置一个进程的就绪队列，即低级调度算法是固定的、单一的，无法满足系统中不同用户对进程调度策略的不同要求，在多处理机系统中，这种单一调度策略实现机制的缺点更显突出，由此，多级队列调度算法能够在一定程度上弥补这一缺点

- **多级反馈队列**
  - 设置多个就绪队列
  - 每个队列都采用FCFS算法
  - 按队列优先级调度
  - 满足各种类型用户的需要:
    - 终端型用户
    - 短批处理作业用户
    - 长批处理作业用户
- **基于公平原则的调度算法**
  - 保证调度算法：明确的性能保证，该算法可以做到调度的公平性
    - 跟踪计算每个进程自创建以来已经执行的处理时间
    - 计算每个进程应获得的处理机时间，即自创建以来的时间除以n
    - 计算进程获得处理机时间的比率，即进程实际执行的处理时间和应获得的处理机时间之比
    - 比较各进程获得处理机时间的比率。如进程A的比率最低，为0.5，而进程B的比率为0.8，进程C的比率为1.2等
    - 调度程序应选择比率最小的进程将处理机分配给它，并让该进程一直运行，直到超过最接近它的进程比率为止。
  - 公平分享调度算法：分配给每个进程相同的处理机时间

3.2死锁

- **引起死锁发生**：需要采用互斥访问方法的、不可以被抢占的资源，即在前面介绍的临界资源。系统中这类资源有很多，如打印机、数据文件、队列、信号量等
- **可重用性资源**：可重用性资源是一种可供用户重复使用多次的资源
  - 每一个可重用性资源中的单元只能分配给一个进程使用，不允许多个进程共享
  - 进程在使用可重用性资源时，须按照这样的顺序
    - 请求资源。如果请求资源失败，请求进程将会被阻塞或循环等待

        - 使用资源。进程对资源进行操作，如用打印机进行打印
        - 释放资源。当进程使用完后自己释放资源
    - 系统中每一类可重用性资源中的单元数目是相对固定的，进程在运行期间既不能创建也不能删除它
- **可消耗性资源=临时资源：**在进程运行期间，由进程动态地创建和消耗的
    - 每一类可消耗性资源的单元数目在进程运行期间是可以不断变化的，有时它可以有许多，有时可能为0
    - 进程在运行过程中，可以不断地创造可消耗性资源的单元，将它们放入该资源类的缓冲区中，以增加该资源类的单元数目
    - 进程在运行过程中，可以请求若干个可消耗性资源单元，用于进程自己的消耗，不再将它们返回给该资源类中
- **可抢占性资源**：某进程在获得这类资源后，该资源可以再被其它进程或系统抢占
- **不可抢占性资源：**一旦系统把某资源分配给该进程后，就不能将它强行收回，只能在进程用完后自行释放
- **计算机系统中的死锁：**
    - 竞争不可抢占性资源引起死锁：系统中所拥有的不可抢占性资源其数量不足以满足多个进程运行的需要，使得进程在运行过程中，会因争夺资源而陷入僵局
    - 竞争可消耗资源引起死锁
    - 进程推进顺序不当引起死锁：进程在运行过程中，对资源进行申请和释放的顺序是否合法
- **产生死锁的必要条件:**
    - 互斥条件
    - 请求和保持条件
    - 不可抢占条件
    - 循环等待条件
- **处理死锁的方法**
    - 预防死锁：通过破坏产生死锁的四个必要条件中的一个或几个，以避免发生死锁，主要是破坏产生死锁的后三个条件
        - 破坏“请求和保持”条件
            - 第一种协议：所有进程在开始运行之前，必须一次性地申请其在整个运行过程中所需的全部资源
            - 第二种协议：允许一个进程只获得运行初期所需要资源，运行过程中边释放已用完资源，边请求新的所需资源
        - 破坏“不可抢占”条件：使其申请新资源不能满足，从而必须释放已经保持的所有资源
        - 破坏“循环等待”条件：系统所有资源类型进行线性排序，并赋予不同的序号
    - 避免死锁：事先预防策略；在资源动态分配过程中，防止系统进入不安全状态，以避免发生死锁；**施加的限制条件较弱，可能获得较好的系统性能，目前常用此方法来避免发生死锁**
        - 安全状态：允许进程动态地申请资源，但系统在进行资源分配之前，应先计算此次资源分配的安全性
        - 安全序列：满足每个进程对资源的最大需求，使每个进程都可以顺利地完成
        - 不安全状态：无法找到这样一个安全序列
    - 检测死锁：检测系统状态，以确定系统中是否发生了死锁
        - 系统死锁，可利用**资源分配图**来描述
        - 死锁定理：找出一个不阻塞也不是独立的点，去除它的请求边和分配边，若能消去图中所有的边，使所有的进程结点都成为孤立结点，则称该图是可完全简化的



- 解除死锁：当认定系统中已发生了死锁，利用该算法可将系统从死锁状态中解脱出来
  - 终止进程的方法
    - 终止所有死锁进程：代价可能会很大
    - 逐个终止进程：按照某种顺序，逐个地终止进程，直至有足够的资源，以打破循环等待，把系统从死锁状态解脱出来为止

3.2.1避免死锁算法

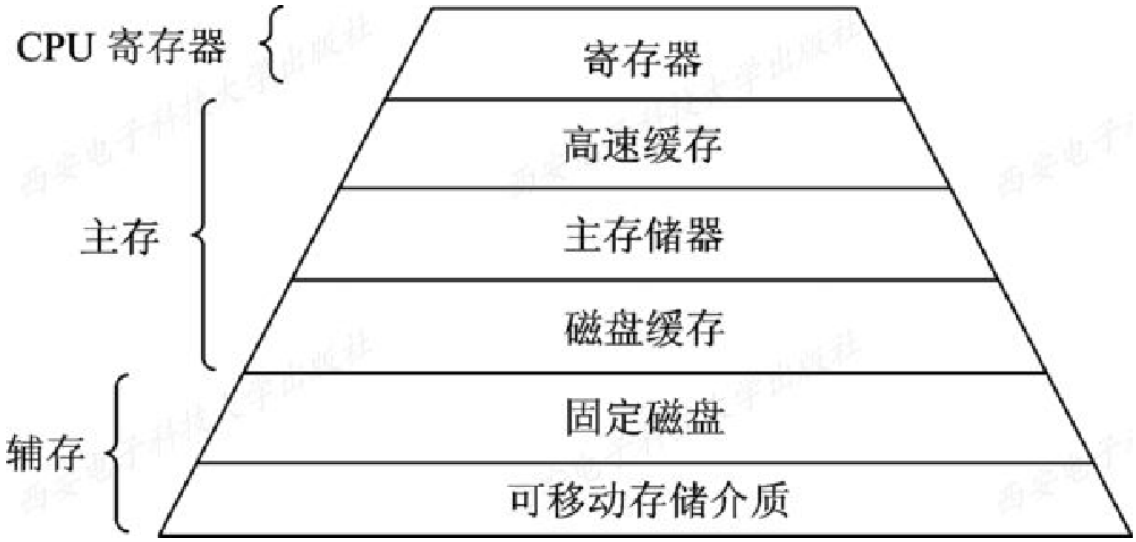
利用银行家算法避免死锁

- 可利用资源向量Available。Available[j] =K
- 最大需求矩阵Max。Max[i, j] = K
- 分配矩阵Allocation。Allocation[i, j] = K
- 需求矩阵Need。Need[i, j] = K

四、存储器管理

4.1存储器的层次结构

- 对于通用计算机而言，存储层次至少应具有三级：最高层为CPU寄存器，中间为主存，最底层是辅存



- 可执行存储器：**寄存器和主存储器**
  - 主存储器=内存=主存：用于保存进程运行时的程序和数据
  - 寄存器：寄存器具有与处理机相同的速度，故对寄存器的访问速度最快，完全能与CPU协调工作，但价格却十分昂贵，因此容量不可能做得很大
- 高速缓存：内存介于寄存器和存储器之间，备份主存中的常用数据，减少处理机对主存的访问次数，大幅提高程序执行速度，访问速度快于主存
- 磁盘缓存：暂时存放频繁使用的一部分磁盘数据和信息，以减少访问磁盘的次数，**并不是真实存在的**，利用主存中的部分存储空间暂时存放从磁盘中读出(或写入)的信息

4.2程序的装入和链接

需要经过以下步骤

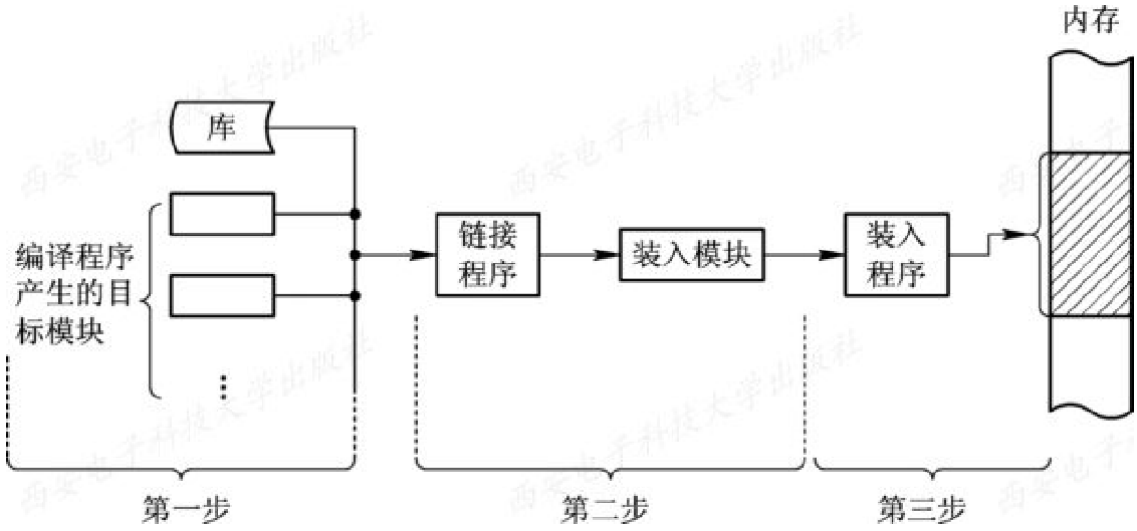
- **编译**
- **链接**
  - 静态链接：在程序运行之前，先将各目标模块及它们所需的库函数链接成一个完整的装配模块，以后不再拆开

    需要解决一下问题：

    - 对相对地址进行修改
    - 变换外部调用符号
  - 装入时动态链接：这是指将用户源程序编译后所得到的一组目标模块，在装入内存时，采用边装入边链接的链接方式

    利点：

    - 便于修改和更新
    - 便于实现对目标模块的共享
  - 运行时动态链接：将所有可能要运行到的模块全部都装入内存，并在装入时全部链接在一起，**低效**，因为有部分目标模块不会运行
- **装入**
  - 绝对装入方式：计算机系统很小，且仅能运行单道程序时
  - 可重定位装入方式：只能将目标模块装入到内存中事先指定的位置，这只适用于单道程序环境
  - 动态运行时的装入方式：可将装入模块装入到内存中任何允许的位置，但是不允许程序运行时在内存中移动位置



4.3连续分配存储管理方式

- 单一连续分配：仅将内存分为系统区（OS使用）和用户区（只有一道用户程序）
- 固定分区分配：通常将分区按其大小进行排队
  - 划分方法：
    - 分区大小相等
    - 分区大小不等
- 动态分区分配
  - 分配中的数据结构
    - 空闲分配区
    - 空闲分配链

4.3.1动态分区分配算法

- 基于顺序搜索的动态分区分配算法
  - 首次适应(first fit，FF)算法：空闲分区链以地址递增的次序链接
  - 循环首次适应(next fit，NF)算法：分配内存空间时，不再是每次都从链首开始查找，而是从上次找到的空闲分区的下一个空闲分区开始查找
  - 最佳适应(best fit，BF)算法：将所有的空闲分区按其容量以从小到大的顺序形成一空闲分区链
  - 最坏适应(worst fit，WF)算法：挑选一个最大的空闲区，从中分割一部分存储空间给作业使用
- 基于索引搜索的动态分区分配算法
  - 快速适应(quick fit)算法=分类搜索法：将空闲分区根据其容量大小进行分类
  - 伙伴系统：无论已分配分区或空闲分区，其大小均为2的k次幂(k为整数，l≤k≤m)
  - 哈希算法：利用哈希快速查找的有点，以及空闲分区在可利用空闲区中表中的分布规律，建立哈希函数，构建一张以空闲分区大小为关键字的哈希表，该表的每一个表项记录了一个对应的空闲分区链表表头指针
- 动态可重新定位分区分配
  - 紧凑：一个系统或用户程序必须被装入一片连续的内存空间中
  - 动态重定位：程序在执行时，真正访问的内存地址是相对地址与重定位寄存器中的地址相加而形成的
  - 动态重定位分区分配算法：与动态分区分配算法基本上相同，这种算法中仅仅只加了**紧凑**的功能

4.4对换

- 引入：减少对系统资源的浪费，提高系统吞吐量
- 类型：
  - 整体对换
  - 页面（分段对换）

在具有对换功能的OS中，通常把磁盘空间分为文件区和对换区两部分，对换分区的分配采用的是连续分配方式，因而对换空间的分配与回收与动态分区方式时的内存分配与回收方法雷同，

4.5分页存储管理方式

- 分页存储管理方式
- 分段存储管理方式
- 段页式存储管理方式

页表功能是由一组专门的寄存器来实现的，一个页表项用一个寄存器，页表存储在内存中。

- 基本分页存储管理中访问内存的有效时间:

$$EAT = t1 + t2 = 2t$$

分别为第一次访问时间和第二次访问时间

- 引入快表的分页存储管理方式中，有效访问时间：

$$EAT = a \times \lambda + (t + \lambda)(1 - a) + t = 2t + \lambda - t \times a$$

λ表示查找快表所需要的时间，a表示命中率，t表示访问一次内存所需要的时间

五、虚拟存储器

5.1基本概念

- **实存管理：**一次性，驻留性（连续内页分配，基本分页分段内存分配）
  - 进程空间不能超过实际内存大小
  - 同时存在的进程数目受限较大
- 虚拟存储器：对内存的虚拟，实际不存在；具有多次性，对换性，虚拟性
- 虚拟存储器系统：具有请求和调入功能和置换功能，从而在逻辑上对内存扩充；只需要进程的一部分装入内存·便可以运行进程
- 容量取决于：计算机的地址结构和外存空间
- 特征：
  - 局部性特征：执行与访问区域有限
  - 空间局部性：该位置被访问则附近位置很快会被访问
  - 时间局部性：被访问过的位置很快会被再次访问
- 管理策略：
  - 调入策略：从外存调入内存（请求调入算法，先行调入算法）
  - 分配策略：如何放入内存（页式和段式内存分配）
  - 淘汰策略：换出哪些内存空间

5.2请求分页存储管理方式

- 页表是主要的数据结构
- 访问的页面不在内存——缺页中断
  - 与一般中断共同点：
    - 保护CPU环境（中断现场）
    - 分析中断原因
    - 缺页中断处理
    - 恢复CPU环境（中断现场）
  - 区别：
    - 缺页中断可以发生在指令执行过程中
    - 一条指令执行期间可能多次发生缺页中断
- 最小物理块数

  取决于指令格式、寻址方式：

  1.单字节单地址且直接寻址需要2个物理块

  2.2字节、2地址指令，最小需6个物理块
- 物理块分配算法：
  - 平均分配算法：同字面，平均分配，不公平
  - 按比例分配算法：按进程大小分配
  - 考虑优先权的分配算法：分两份一份按比例分配，一份根据优先权，优先级高的多分配：
- 可变页面分配：缺页中断时，再分配新的物理块
- 局部页面置换策略：缺页时候，从本进程中，按一定算法选择一页调出
- 全局页面置换策略：缺页时候采用：可变页面，没有新的采用局部页面
- 页面调入策略：
  - 预调页策略：一次调入多个相邻页
  - 请求调页策略：调入一页

如页面足够则全从对换区调入，页面紧张：先从文件区调入页面，如果该页面被修改，则换出时，放到对换区；再调入时，则从对换区调入；未被修改的页面仍从文件区调入。

unix方法：位运行从文件区调入，反之对换区

5.3抖动

随着进程数的增加，处理机的利用率反而下降

产生原因：每个进程的大部分时间都用于页面的换进/换出。